Perbandingan Hasil Optimasi Algoritma Kompresi Huffman dan LZW dengan Metode Transformasi Burrows Wheeler

PROPOSAL TUGAS AKHIR

oleh :

Jonathan Marcel T/13507072



Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
INSTITUT TEKNOLOGI BANDUNG
Desember 2010

1 Ringkasan Proposal

Proposal tugas akhir ini berisikan hal-hal penting yang akan melandasi pembuatan tugas akhir pada semester II/2010-2011. Pada proposal ini diajukan sebuah topik yaitu “Perbandingan Hasil Optimasi Algoritma Kompresi Huffman dan LZW dengan Metode Transformasi Burrows Wheeler. Proposal ini mencakup banyak bagian, antara lain latar belakang, rumusan masalah, tujuan, studi literatur, metodologi, implikasi, struktur laporan, penjadwalan, dan referensi.

Pada bagian pendahuluan dijelaskan secara singkat bahwa metode pemampatan data perlu dioptimasi dikarenakan kebutuhan manusia dalam menyimpan data dan informasi tidak sebanding dengan peningkatan kapasitas media penyimpanan yang tersedia. Pada bagian rumusan masalah dan tujuan dijelaskan mengenai masalah apa yang melandasi pembuatan tugas akhir ini, yaitu bagaimana perbandingan hasil optimasi algoritma Huffman dan LZW serta apakah tujuan dari optimasi tersebut. Bagian berikutnya ialah studi literatur yang berisi kajian-kajian yang diperoleh dari literatur yang sekiranya relevan dengan tugas akhir ini. Bagian ini berisi beberapa teori antara lain teori data dan informasi, teori pemampatan data, algoritma Huffman, algoritma LZW, maupun transformasi Burrows Wheeler. Bagian selanjutnya ialah metodologi yang berisikan metode penelitian yang akan digunakan, antara lain studi literatur, analisis, perancangan dan implementasi, pengujian dan evaluasi, hingga pengambilan keputusan dan saran. Selanjutnya bagian implikasi yang berisi apa kontribusi dari tugas akhir terhadap perkembangan ilmu komputer. Lalu terdapat struktur laporan yang berisi sistematika dari laporan tugas akhir yang akan dibuat, kurang lebih urutannya adalah pendahuluan, dasar teori, analisis, perancangan dan implementasi, pengujian dan evaluasi, serta penutup. Kemudian terdapat penjadwalan yang berisikan penjadwalan pelaksanaan tugas akhir yang telah didiskusikan dengan pembimbing dan disesuaikan dengan target kelulusan, yaitu bulan Juli 2011, serta yang terakhir ialah referensi yaitu berisikan daftar seluruh referensi yang digunakan dalam pembuatan tugas akhir ini.

2 Latar Belakang

Sejak dahulu kala, informasi merupakan salah satu kebutuhan manusia yang paling mendasar. Salah satu penyebab mengapa informasi ini sangatlah penting karena manusia memiliki kodrat sebagai makhluk sosial. Pada dasarnya manusia memerlukan informasi untuk menentukan tindakan apa yang harus dilakukan. Informasi juga merupakan hal yang vital di dalam proses komunikasi.

Data merupakan informasi yang belum diolah, artinya data ini hanya merupakan kumpulan fakta yang dapat berupa angka, karakter, simbol, gambar, dan lain sebagainya. Data ini merupakan deskripsi dari sesuatu kejadian yang dihadapi. Data yang telah diproses menjadi sesuatu yang memiliki makna maka akan disebut sebagai informasi[HAN03]. Kebanyakan sumber informasi menyediakan informasi masih dalam bentuk data, dan kemudian akan diolah di otak manusia yang menerimanya sehingga menjadi informasi yang bermanfaat.

Data maupun informasi akan disimpan atau diarsip sehingga dapat digunakan kembali di kemudian hari. Penyimpanan data atau informasi dapat menggunakan media penyimpanan, misalnya *notes*, *harddisk*, *flashdisk*, CD/DVD, maupun dengan menggunakan media penyimpanan lainnya.

Seiring dengan perkembangan teknologi, pertumbuhan jumlah data maupun informasi berkembang lebih pesat dibandingkan dengan kapasitas media penyimpanan. Karena keterbatasan kapasitas penyimpanan tersebut, maka muncul gagasan untuk menyimpan data atau informasi tersebut dengan lebih efisien[WIT99]. Dengan menyimpan data secara lebih efisien, maka manusia dapat menghemat ruang penyimpanan sehingga jumlah data yang disimpan bisa lebih banyak. Salah satu solusi untuk menyimpan data secara lebih efisien ialah dengan menggunakan teknik kompresi data.

Ide dasar dari pemampatan data ialah banyaknya redundansi (pengulangan) pada representasi data. Pada dasarnya, kompresi data atau pemampatan data ialah memampatkan ukuran data sehingga menjadi lebih kecil dibandingkan

sebelumnya. Hal ini bisa dilakukan karena ukuran data akan di-*encode* menjadi bit-bit yang lebih kecil dengan menggunakan metode tertentu.

Ada dua jenis metode pemampatan data yang sering digunakan, yaitu metode statistik dan metode kamus[SAL07]. Pada pemampatan data dengan metode statistik, digunakan *variable-size code*, yaitu mengkodekan setiap karakter dengan panjang yang berbeda-beda, dimana karakter yang sering muncul di dalam dokumen akan dikodekan menjadi berukuran lebih pendek. Sedangkan pada pemampatan data dengan metode kamus, digunakan sebuah struktur data yaitu kamus, yang berfungsi untuk menyimpan karakter-karakter yang terdapat di dalam dokumen. Karakter yang belum terdapat di dalam dokumen akan dimasukkan ke dalam kamus, sehingga pada saat pemampatan, cukup dipanggil pointer pada kamus tersebut sesuai dengan karakter yang ditunjukkan.

Ada banyak algoritma pemampatan data secara statistik, diantaranya yang terkenal antara lain: Shannon Fano Coding, Huffman Coding, Adaptive Huffman Coding, MNP7, Arithmetic Coding, QM Coder, dan PPM. Sedangkan beberapa contoh algoritma pemampatan data dengan kamus yang terkenal antara lain: LZ77 (*Sliding Window)*, LZX, QIC-122, LZ78, LZW, LZAP, Deflate, LZMA, kompresi XML, dan CRC.

Untuk mengoptimasi algoritma pemampatan data, salah satu metode yang digunakan adalah metode transformasi Burrows Wheeler[SIM08]. Optimasi algoritma pemampatan data bertujuan untuk menghasilkan data yang dimampatkan secara lebih cepat maupun dengan ukuran yang lebih kecil dibandingkan dengan kinerja algoritma sebelum dioptimasi. Metode transformasi Burrows Wheeler mentransformasikan isi dari file teks berdasarkan permutasi seluruh kemungkinan rotasi dari karakter-karakter di dalam file teks. Hasilnya ialah file teks yang akan dimampatkan mengalami perubahan bentuk, dimana bagian-bagian yang memiliki kemiripan akan ditempatkan secara berdekatan.

Pada tugas akhir ini, akan dibandingkan bagaimana kinerja dua algoritma pemampatan data yang cukup populer, yaitu algoritma Huffman, yang mewakili

metode statistik, dan algoritma LZW, yang mewakili metode kamus. Selain itu, kedua algoritma tersebut akan mengalami optimasi dengan metode transformasi Burrows Wheeler, sehingga hasil pemampatan datanya secara teoritis akan menjadi lebih baik. Untuk proses perbandingan hasil optimasinya, kedua algoritma akan diimplementasi ke dalam suatu perangkat lunak sederhana, sehingga memudahkan proses pengujian dan evaluasi terhadap kinerja kedua algoritma sebelum maupun sesudah dioptimasi.

3 Rumusan Masalah

Ide yang mendasari pengerjaan tugas akhir ini ialah untuk melakukan perbandingan kinerja hasil optimasi terhadap dua buah algoritma kompresi data yang cukup popular, yaitu algoritma Huffman dan algoritma Lempel Ziv Welch (LZW). Optimasi algoritma diharapkan dapat menghasilkan data hasil kompresi yang lebih termampatkan.

Secara rinci, tahapan dari rumusan masalah di atas ialah:

1. Bagaimana melakukan optimasi terhadap kinerja algoritma Huffman dengan menggunakan metode transformasi Burrows Wheeler.
2. Bagaimana melakukan optimasi terhadap kinerja algoritma LZW dengan menggunakan metode transformasi Burrows Wheeler.
3. Bagaimana melakukan pengukuran tingkat kinerja algoritma Huffman pada saat pemampatan data baik sebelum maupun setelah mengalami optimisasi.
4. Bagaimana melakukan pengukuran tingkat kinerja algoritma LZW pada saat pemampatan data baik sebelum maupun setelah mengalami optimisasi.
5. Bagaimana membandingkan kinerja kedua algoritma tersebut sebelum maupun setelah mengalami optimisasi.

4 Tujuan

Tujuan yang ingin dicapai dari pengerjaan tugas akhir ini ialah membandingkan kinerja algoritma LZW dan Huffman sebelum dan sesudah dioptimasi sehingga dapat diketahui manakah diantara kedua metode tersebut yang menghasilkan rasio kompresi yang lebih baik.

Adapun tahapan untuk mencapai tujuan tersebut ialah:

1. Mempelajari dan memahami algoritma kompresi data Huffman, algoritma LZW, dan transformasi Burrows Wheeler.
2. Melakukan optimasi terhadap kinerja algoritma Huffman dan LZW dengan menggunakan transformasi Burrows Wheeler.
3. Melakukan implementasi algoritma Huffman, LZW, dan transformasi Burrows Wheeler untuk melakukan pengujian kinerja algoritma.
4. Membandingkan hasil pengujian kinerja kedua algoritma sebelum maupun sesudah dioptimasi dengan transformasi Burrows Wheeler.

5 Studi Literatur

5.1 Teori Data dan Informasi

Pada dasarnya data dapat diartikan sebagai representasi dari fakta. Awalnya, data hanyalah merupakan kumpulan fakta yang belum memiliki makna. Untuk memperoleh makna dari suatu data, data tersebut harus mengalami pemrosesan, sehingga akan berubah menjadi informasi[WIT99].

Teori informasi yang dikembangkan oleh Claude E. Shannon mempelajari aspek kuantitatif dari informasi[HAN03]. Dengan menggunakan teori ini, dapat dihitung banyaknya informasi yang terdapat dalam suatu data dengan angka yang tepat. Karena itulah didefinisikan unit dari informasi yang disebut dengan bit. Saat ini sudah menjadi hal yang lumrah untuk menyebutkan ukuran dari suatu informasi dalam satuan bit. Teori informasi banyak diaplikasikan di berbagai bidang, diantaranya pemrosesan bahasa alami, kriptografi, komputasi kuantum, inferensi statistik, dan lain-lain.

Dalam ilmu komputer, data digunakan sebagai informasi yang dibutuhkan oleh komputer agar dapat menjalankan fungsinya. Di dalam komputer, data disimpan dalam format *binary* dan direpresentasikan oleh kumpulan angka 0 dan 1. Setiap angka 0 maupun 1 menghabiskan ruang sebesar 1 bit di dalam media penyimpanan.

Jumlah data yang terus bertambah tidak diimbangi dengan peningkatan kapasitas media penyimpanan data. Oleh karena itu, manusia mencari cara agar dapat menyimpan data secara lebih efektif untuk memanfaatkan kapasitas media penyimpanan secara optimal. Salah satu gagasan yang muncul ialah bagaimana merepresentasikan data dalam bentuk yang lebih ringkas, sehingga data dapat disimpan dalam ukuran yang lebih kecil[HAN03].

5.2 Teori Pemampatan Data

Pemampatan data dapat diartikan sebagai proses mengubah aliran data masukan ke dalam aliran data lain yang lebih kecil. Pemampatan data, atau disebut juga

kompresi data memiliki berbagai matode berdasarkan ide yang berbeda-beda. Proses pemampatan data menjadi populer dalam perkembangan ilmu komputer dikarenakan oleh hal-hal berikut[SAL07]:

1. Manusia memiliki kecenderungan untuk mengumpulkan data secara terus-menerus tanpa menghapus data sebelumnya. Hal ini tentunya menjadi masalah karena kapasitas media penyimpanan semakin lama semakin berkurang. Akibatnya, data baru yang akan disimpan tidak lagi memiliki tempat penyimpanan. Dengan menggunakan metode pemampatan data, jumlah data yang disimpan dalam media penyimpanan dapat ditingkatkan.
2. Waktu transfer data dapat dipercepat apabila data berukuran lebih kecil. Hal ini sangat bermanfaat, terutama karena pada era internet sekarang, kecepatan transfer data menjadi *constraint* yang sangat penting untuk para pengguna internet. Dengan menggunakan metode pemampatan data yang sesuai, data yang akan ditransfer dapat dikurangi ukurannya dan menyebabkan waktu transfer data menjadi berkurang.

Prinsip dasar metode pemampatan data ialah dengan menyingkirkan bagian yang dianggap redundan dari aliran data masukan. Hal ini berdasarkan prinsip bahwa data memiliki struktur tertentu, dimana struktur ini dapat dimanipulasi untuk mendapatkan representasi data dalam ukuran yang lebih kecil. Karena struktur data dapat berubah antara berbagai jenis data, maka tidak ada algoritma pemampatan data yang bekerja secara optimal untuk seluruh jenis data.

Metode pemampatan data dapat dibagi menjadi dua bagian, yaitu pemampatan data lossless dan pemampatan data lossy[SAL07]. Pada pemampatan data lossy, data yang dimampatkan akan memiliki perbedaan dengan data asal karena pada metode ini sebagian informasi pada data asal akan dihilangkan. Metode pemampatan data lossy banyak digunakan untuk memampatkan data video, suara, dan gambar. Hal ini dikarenakan pada tipe data tersebut, tidak akan terlihat perbedaan yang signifikan apabila sebagian informasi dari tipe data tersebut dihilangkan. Namun lain halnya pada data teks, dimana perubahan informasi,

bahkan hanya sebesar 1 bit, dapat mengakibatkan perbedaan yang signifikan antara data awal dan data hasil pemampatan. Karena itulah, pada pemampatan data teks sebaiknya menggunakan metode lossless, yaitu data asal sama dengan data hasil pemampatan.

Beberapa hal yang dapat digunakan untuk mengukur kinerja dari sebuah algoritma pemampatan data ialah sebagai berikut[SAL07]:

1. Rasio pemampatan data (*compression ratio*)

Adalah ukuran hasil pemampatan data dibagi dengan ukuran file semula. Semakin kecil nilai rasio pemampatan data, semakin efektif kinerja algoritma kompresi tersebut. Sebagai contoh, apabila nilai rasio 0,7 berarti algoritma menghasilkan data keluaran berukuran 70% dari ukuran semula.

$$R_o = \frac{Ukuran\ \ keluaran\ \ aliran\ \ data}{Ukuran\ \ masukan\ \ aliran\ \ data}$$

2. Faktor pemampatan data (*compression factor*)

Adalah inverse dari rasio pemampatan data. Apabila nilainya lebih besar dari satu, maka algoritma pemampatan data tersebut tidak layak untuk digunakan.. Karena apabila nilai lebih dari satu maka terjadi ekspansi data, dan nilai kurang dari satu maka terjadi pemampatan data.

$$F_o = \frac{Ukuran\ \ masukan\ \ aliran\ \ data}{Ukuran\ \ keluaran\ \ aliran\ \ data}$$

3. Kecepatan kompresi (*compression speed*)

Adalah perbandingan ukuran berkas file yang akan dimampatkan dengan waktu yang dibutuhkan untuk memampatkan berkas file tersebut. Apabila kecepatan kompresi kecil maka waktu yang dibutuhkan untuk memampatkan data menjadi lama sehingga algoritma pemampatan data tersebut tidak banyak digunakan.

$$\text{Kecepatan kompresi} = \frac{Ukuran\ \ file\ \ semula}{Waktu\ \ kompresi}$$

4. Kecepatan dekompresi (*decompression speed*)

Adalah perbandingan ukuran berkas file hasil dimampatkan dengan waktu yang dibutuhkan untuk mendekompresi berkas file tersebut menjadi bentuk file awal. Algoritma pemampatan data yang baik haruslah memiliki kecepatan kompresi dan dekompresi yang saling berimbang.

$$\text{Kecepatan dekompresi} = \frac{Ukuran\ \ file\ \ hasil\ \ pemampatan}{Waktu\ \ dekompresi}$$

Setelah itu pemampatan data mulai memanfaatkan redundansi pada representasi karakter ASCII. Representasi karakter dengan ASCII memiliki redundansi yang besar, karena ASCII mengkodekan setiap karakter dengan panjang yang tetap. Dengan memanfaatkan redundansi tersebut, muncul gagasan untuk melakukan pengkodean karakter dengan panjang bit variatif, yang dikenal dengan sebutan *varibale size code*.

Untuk mendapatkan kinerja yang baik dari pengkodean bit dengan panjang variatif, maka diperlukan pengetahuan tentang jumlah kemunculan setiap karakter dalam suatu berkas file, agar setiap karakter dapat direpresentasikan dengan panjang kode yang berbeda. Karakter yang memiliki tingkat kemunculan tinggi direpresentasikan dengan kode yang lebih pendek, sedangkan karakter yang memiliki tingkat kemunculan rendah direpresentasikan dengan kode yang lebih panjang. Itulah mengapa pendekatan statistik digunakan, yaitu untuk menghitung kemunculan setiap karakter pada file. Salah satu pendekatan statistik yang populer ialah metode pemampatan data Huffman.

Karena pendekatan secara statistik sangat bergantung kepada permodelan statistik yang dibuat, maka manusia menciptakan pendekatan lain, yaitu pendekatan dengan menggunakan kamus. Pendekatan dengan menggunakan kamus tidak bergantung pada model statistik dan tidak menggunakan kode dengan panjang variatif (*variable size code*). Pada pendekatan ini yang dilakukan ialah membangun sebuah kamus berdasarkan berkas masukan, dan kamus ini berguna untuk melakukan encoding string masukan menjadi token. Kamus ini dapat

bersifat statis, yaitu kamus mengizinkan penambahan simbol saja, maupun bersifat adaptif, yaitu kamus mengizinkan penambahan simbol maupun pengurangan simbol dari dalam kamus.

Langkah-langkah umum pada pemampatan data dengan menggunakan pendekatan kamus ialah:

1. Melakukan pembacaan teks
2. Membuat kamus berdasarkan kemunculan kata pada berkas
3. Mengkodekan setiap kata berdasarkan indeks lokasi kata pada kamus

5.3 Metode pemampatan data Huffman (Huffman Coding)

Huffman Coding merupakan salah satu algoritma dengan menggunakan metode statistik. Hal ini disebabkan karena huffman coding memampatkan data berdasarkan jumlah kemunculan kata dari setiap karakter. Huffman Coding merupakan salah satu metode pemampatan data yang sangat popular karena banyak digunakan sebagai basis pada berbagai program di berbagai *platform.* Huffman Coding memiliki kemiripan dengan metode pemampatan data Shannon-Fano (Shannon-Fano Coding)[SAL07]. Perbedaannya ialah, Huffman Coding mengkonstruksi pohon kode dari bawah ke atas, atau dengan kata lain mengkodekan simbol dari kanan ke kiri, sedangkan pada Shannon-Fano Coding ialah kebalikannya, yaitu membuat pohon kode dari atas ke bawah, atau dengan kata lain mengkodekan simbol dari bit paling kiri hingga bit paling kanan.

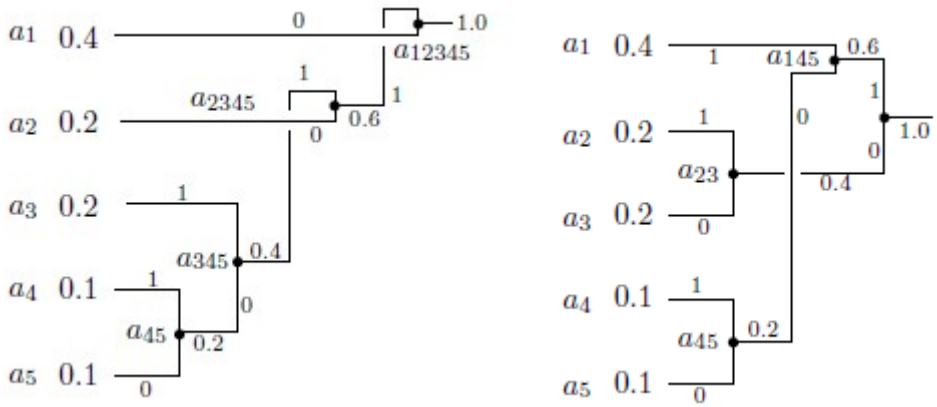
Algoritma dasar dari Huffman Coding ialah menggabungkan dua karakter yang paling tidak sering muncul pada berkas, dan mengulang proses ini hingga tersisa sebuah karakter saja[HUF52]. Pada saat penggabungan ini, dibentuk struktur pohon kode dan kode Huffman akan diperolah dari proses *labeling* pada pohon kode tersebut.

Contoh dari algoritma Huffman dapat dijabarkan sebagai berikut:

Diberikan 5 simbol a_1, a_2, a_3, a_4, dan a_5, dengan probabilitas masing-masing 0.4, 0.2, 0.2, 0.1, dan 0.1. Simbol-simbol tersebut akan dipasangkan dengan urutan sebagai berikut:

1. a_4 digabungkan dengan a_5 dan keduanya diganti dengan simbol bantu a_{45}, dengan probabilitas 0.2
2. Sekarang terdapat 4 simbol tersisa, a_1, dengan probabilitas 0.4, a_2, a_3, a_{45}, dengan probabilitas masing-masing 0.2. Dengan memilih acak a_3 dan a_{45}, kemudian digabungkan, dan diganti dengan simbol bantu a_{345}, dengan probabilitas 0.4.
3. Sekarang tinggal 3 simbol yang tersisa, a_1, a_2, dan a_{345}, dengan probabilitas 0.4, 0.2, dan 0.4. Dengan memilih secara random a_2 dan a_{345}, kemudiang digabungkan, dan diganti simbol bantu a_{2345} dengan probabilitas 0.6.
4. Akhirnya, digabungkan dua simbol tersisa yaitu a_1 dan a_{2345} dan diganti dengan simbol bantu a_{12345} dengan probabilitas 1.

Sekarang pohon sudah selesai dibentuk. Pohon yang dibentuk dapat dilihat pada gambar kiri.



Dalam proses pengkodean, untuk setiap busur diberikan bit 1 untuk sisi atas dan bit 0 untuk sisi bawah. Hasilnya adalah simbol a_1 dikodekan sebagai 0, simbol a_2 dikodekan sebagai 10, simbol a_3 dikodekan sebagai 111, simbol a_4 dikodekan sebagai 1101, dan simbol a_5 dikodekan sebagai 1100.

Yang perlu diingat bahwa kode pada algoritma Huffman tidaklah unik. Perhatikan gambar kanan, pada gambar tersebut proses pembentukan pohon memiliki perbedaan dengan pembentukan pohon pada gambar kiri. Gambar tersebut menunjukkan bahwa kelima simbol tersebut digabungkan secara berbeda menghasilkan kode yang berbeda pula, yaitu 11, 01, 00, 101, dan 100. Akan tetapi hasil penggabungannya haruslah menyisakan sebuah karakter dengan angka probabilitas sama dengan satu.

Beberapa hal yang perlu diperhatikan pada algoritma pemampatan data Huffman:

1. Tidak ada ketentuan bagaimana mengurutkan simbol-simbol alfabet, namun sebaiknya pengurutan didasarkan pada jumlah kemunculan setiap simbol pada file.
2. Tidak ada ketentuan bagaimana proses labeling pada busur pohon, ada dua cara yang bisa digunakan yaitu, sisi atas busur diberi bit 0 dan sisi bawah busur diberi bit 1 ataupun sebaliknya. Yang penting ialah konsistensi pemberian label pada busur pohon.
3. Kode pada algoritma Huffman tidaklah unik.
4. Kode pada algoritma Huffman optimal jika tidak ada kode-kode lain dengan panjang berbeda yang memiliki tingkat rata-rata panjang bit/karakter yang lebih rendah. Artinya ialah apabila salah satu pengkodean pada suatu file teks menghasilkan nilai rata-rata 2.2 bit/karakter, maka bentuk pengkodean lainnya harus menghasilkan nilai rata-rata 2.2 bit/karakter.

5.4 Metode Pemampatan Data Lempel Ziv Welch (LZW)

Algoritma pemampatan data Lempel Ziv Welch adalah algoritma pemampatan data dengan menggunakan pendekatan kamus[ZIV77]. Kamus ini digunakan untuk menyimpan string yang telah ditemukan sebelumnya. Algoritma yang ditemukan oleh Abraham Lempel, Jacob Ziv, dan Terry Welch ini dipublikasikan sebagai implementasi yang lebih baik dari algoritma LZ78.

Untuk memahami algoritma LZW secara lebih mendalam, pertama-tama akan dibahas perbandingan algoritma LZW dengan algoritma LZ78. Pada awalnya kamus pada algoritma LZ78 kosong dan ukurannya terbatas oleh memori yang dapat digunakan. Pemampatan data dengan menggunakan algoritma LZ78 ialah sebagai berikut[SAL07]:

1. Kamus diawali dengan karakter null yang memiliki indeks 0
2. Baca karakter berikutnya pada file input. Setiap string yang ditambahkan pada kamus akan diberi indeks 1, 2, dan seterusnya.
3. Saat karakter x dibaca dari file input, kamus mencari sebuah token yang berisi string x. Token bisa ditulis dalam bentuk (X,Y), dimana X berupa bilangan bulat yang menunjukkan penunjuk pada kamus, dan Y berupa string.
    a) Jika string tidak ditemukan, maka token (0,x) ditambahkan di dalam kamus.
    b) Jika string x ditemukan, misalnya pada posisi k, maka string berikutnya y dibaca, dan kamus mencari token yang berisi xy. Jika tidak ditemukan, maka token (k,y) ditambahkan di dalam kamus. Sedangkan apabila karakter ditemukan, maka proses pembacaan berlanjut terus hingga ditemukan karakter yang tidak berada di dalam kamus.
4. Kemudian ulangi dari langkah kedua hingga seluruh isi file terbaca.

Sedangkan pada algoritma pemampatan data LZW, kamus tidak diinisialisasi dengan karakter null, melainkan telah diisi dengan 0-255 karakter ASCII atau 256 8-bit byte. Perbedaan yang lain ialah, token pada LZW hanya terdiri atas satu elemen, yaitu penunjuk pada kamus, karena kamus pada LZW sudah diinisialisasi sehingga karakter yang akan dibaca berikutnya pasti telah ada di dalam kamus. Prinsip utama algoritma ini ialah memampatkan karakter satu per satu dan mengakumulasikannya ke dalam string I Pemampatan data dengan algoritma LZW adalah sebagai berikut[SAL07]:

1. Inisialisasi kamus dengan 256 entri di dalam kamus
2. Baca karakter berikutnya pada file input.
3. Setiap karakter dikonkatenasi dengan string I.

a) Apabila I terus menerus ditemukan di dalam kamus, maka proses pembacaan terus berlanjut.
b) Apabila konkatenasi karakter dengan string I tidak ditemukan di dalam kamus, misalnya Ix, maka kamus menambahkan token baru di dalam kamus, yaitu penunjuk ke string I, menyimpan string Ix di dalam kamus, dan menginisialisasi string I ke karakter x.

4. Ulangi mulai langkah yang kedua hingga pembacaan file input selesai.

5.5 Transformasi Berkas

Transformasi berkas adalah proses melakukan modifikasi pada berkas sehingga mengubah berkas menjadi bentuk yang lebih mudah untuk dimampatkan. Transformasi berkas biasanya dilakukan sebelum memampatkan data agar bisa mendapatkan hasil pemampatan data yang lebih baik.

Ada 3 jenis transformasi yang seringkali digunakan dalam memproses berkas sebelum dimampatkan, yaitu[RAD09]:

1. Transformasi Burrows Wheeler

Transformasi Burrows Wheeler menggunakan matriks untuk menyusun permutasi kemunculan karakter dalam berkas, sehingga akhirnya pola repetisi yang tadinya terletak secara tersebar dalam berkas teks dapat dikumpulkan pada lokasi yang berdekatan.

2. Transformasi Star

Transformasi Star berdasarkan pada pendefinisian tanda yang unik unuk setiap karakter dan menggantikan karakter tersebut dengan karakter khusus, contohnya karakter *. Transformasi ini menggunakan kamus untuk melakukan transformasi pada berkas.

3. Transformasi LIPT (Length Index Preserving Transform)

Transformasi LIPT merupakan pengembangan dari transformasi Star. Transformasi LIPT menggunakan kamus dan memiliki kecepatan yang lebih baik dibandingkan transformasi Star. Namun sayangnya transformasi ini belum popular digunakan.

5.6 Transformasi Burrows Wheeler

Transformasi Burrows Wheeler adalah salah satu metode yang digunakan untuk melakukan transformasi pada berkas teks. Transformasi yang terjadi ialah menjadikan bagian-bagian yang banyak berulang pada teks menjadi berdekatan. Pada proses transformasi, berkas teks hasil transformasi akan menjadi lebih besar karena ditambahkan variabel lokasi baris pada hasil transformasinya. Bagian yang saling berulang akan diletakkan berdekatan sehingga menjadikan berkas teks tersebut lebih mudah untuk dimampatkan[SIM08].

Cara kerja algoritma transformas Burrows Wheeler adalah sebagai berikut:

1. Membaca string x
2. Membuat tabel yang berisi semua kemungkinan rotasi dari string x
3. Mengurutkan kolom tabel secara alfabetis
4. Mengembalikan kolom terakhir dan nomor baris yang mengandung berkas awal sebagai berkas yang telah ditransformasi

Cara untuk mengembalikan berkas hasil transformasi adalah sebagai berikut:

1. Membuat sebuah tabel kosong
2. Membaca string yang ditransformasi y
3. Ulangi sebanyak panjang y:

a) Masukkan string y sebagai kolom sebelum kolom pertama
b) Lakukan pengurutan secara alfabetis

4. Jadikan baris yang bersesuaian dengan nomor baris yang terdapat pada berkas yang telah ditransformasi sebagai keluaran

5.7 Kajian Terkait

Sebelum mengambil tugas akhir dengan topik ini, telah ditelaah beberapa literatur yang memiliki keterkaitan dengan penelitian di bidang algoritma pemampatan data. Beberapa literatur yang terkait adalah:

1. Tugas akhir Arinta Praimandini Auza, berjudul "Analisis Perbandingan Algoritma LZ78 dan *Arithmetic Coding* pada Kompresi File Teks"

Pada tugas akhir ini saudari Arinta membandingkan kinerja algoritma kompresi LZ78 dan algoritma kompresi *Arithmetic Coding*. Hasil yang diperoleh ialah algoritma *Arithmetic Coding* mengkompresi file menjadi lebih kecil, tetapi algoritma LZ78 lebih unggul dari segi waktu kompresi. Selain itu saudari Arinta menyimpulkan bahwa algoritma LZ78 tidak cocok untuk mengkompresi file berukuran besar, karena semakin besar ukuran file algoritma LZ78 semakin tidak akurat.

2. Tugas akhir Adi Purwanto Sujarwadi, berjudul "Optimasi Kinerja Algoritma Pemampatan Data Lempel-Ziv dengan Metode Transformasi Burrows Wheeler"

Pada tugas akhir ini, saudara Adi melakukan optimasi pada algoritma LZ77 (*sliding window)* dengan menggunakan metode transformasi Burrows Wheeler. Hasil yang diperoleh ialah, preproses teks dengan algoritma Burrows Wheeler dapat meningkatkan kinerja algoritma kompresi LZ77, yaitu menghasilkan file hasil kompresi yang berukuran lebih kecil. Selain itu saudara Adi menyimpulkan bahwa transformasi Burrows Wheeler justru dapat mengubah bentuk file menjadi file yang sulit dikompresi, contohnya ialah file dalam kategori *calgalry corpus.*

Oleh karena itu, tugas akhir yang diambil secara ringkas melanjutkan tugas akhir saudara Adi. Karena tugas akhir saudara Adi menggunakan algoritma LZ77, maka tugas akhir ini menggunakan algoritma LZW yang merupakan penyempurnaan dari algoritma LZ77. Selain itu dilakukan optimasi pada dua jenis algoritma pemampatan data, yaitu algoritma Huffman, yang mewakili metode statistik, dan

algoritma LZW, yang mewakili metode kamus. Kedua-duanya dioptimasi dengan algoritma transformasi Burrows Wheeler.

6 Metodologi

Dalam mengerjakan tugas akhir ini, ada beberapa langkah yang dilakukan, yaitu:

1. Studi Literatur

Pada tahap ini dilakukan pembelajaran mengenai teori dasar yang digunakan pada tugas akhir ini, antara lain tentang teori kompresi data, algoritma Huffman, algoritma LZW, dan transformasi Burrows Wheeler. Literatur yang digunakan antara lain bahan dari situs internet, makalah ilmiah, buku teks, maupun tugas akhir yang relevan dengan topik kompresi data.

2. Analisis

Pada tahap ini dilakukan analisis mengenai optimasi yang dilakukan, antara lain hipotesis optimasi dengan menggunakan transformasi Burrows Wheeler dapat meningkatkan kinerja algoritma kompresi Huffman maupun LZW. Pada tahap ini juga dilakukan analisis terhadap perangkat lunak yang dibangun untuk melakukan pengujian terhadap kinerja kedua algoritma sebelum dan sesudah dioptimasi.

3. Perancangan dan Implementasi

Pada tahap ini dilakukan perancangan serta implementasi algoritma kompresi dalam bentuk perangkat lunak. Perangkat lunak yang dibangun berdasarkan hasil analisis yang telah dilakukan sebelumnya. Perangkat lunak ini berguna untuk membantu mengukur kinerja kedua algoritma sebelum maupun sesudah dioptimasi dengan transformasi Burrows Wheeler.

4. Pengujian dan Evaluasi

Pada tahap ini dilakukan pengujian perangkat lunak yang telah dibuat untuk menguji apakah perangkat lunak bisa berfungsi sesuai rancangan yang telah

dibuat. Setelah itu akan dilakukan evaluasi kinerja algoritma Huffman dan LZW sebelum maupun sesudah dioptimasi.

5. Kesimpulan dan Saran

Pada tahap ini dilakukan penarikan kesimpulan studi yang telah dilakukan. Akan ditarik kesimpulan apakah hipotesis awal penelitian terbukti ataupun tidak terbukti. Kemudian akan diberikan saran bagi penelitian selanjutnya yang memiliki relevansi dengan tugas akhir ini.

7 Implikasi

Implikasi dari tugas akhir ini adalah:

1. Algoritma pemampatan data sudah sangat luas perkembangannya karena manusia berusaha menyimpan data di dalam media penyimpanan yang kapasitasnya terbatas, akibatnya manusia membutuhkan metode pemampatan data yang bekerja secara optimal. Salah satu cara optimasi algoritma pemampatan data yang sudah ada ialah dengan menggunakan metode transformasi Burrows Wheeler.
2. Penelitian yang dilakukan dapat memberikan kontribusi bagi perkembangan ilmu komputer, khususnya dalam bidang pemampatan data. Kontribusi yang diberikan salah satunya ialah modifikasi algoritma Huffman maupun LZW dengan menerapkan metode transformasi Burrows Wheeler.
3. Pada tugas akhir ini optimasi algoritma pemampatan data dilakukan dengan menggunakan berkas file teks. Diharapkan untuk ke depannya berbagai jenis file lain seperti file gambar, file suara, dan file video dapat dimampatkan secara optimal dengan metode tertentu.
4. Meskipun algoritma pemampatan yang sering digunakan sekarang, misalnya algoritma pada perangkat lunak WinRar atau WinZip, telah memiliki kinerja yang cukup baik, tetapi alangkah baiknya apabila dilakukan penelitian dengan algoritma pemampatan data yang merupakan

algoritma-algoritma dasar seperti algoritma Huffman maupun LZW, karena algoritma-algoritma tersebutlah yang menjadi dasar pengembangan algoritma pemampatan data modern seperti Bzip2 dan DEFLATE.

8 Struktur Laporan

Laporan tugas akhir yang akan dibuat memiliki sistematika sebagai berikut: Pendahuluan, dasar teori, analisis, perancangan dan implementasi, pengujian dan evaluasi, serta penutup.

Pendahuluan berisikan latar belakang tugas akhir, rumusan masalah, tujuan, batasan masalah, metodologi pengerjaan tugas akhir dan sistematika laporan tugas akhir.

Dasar teori berisikan teori-teori yang menjadi dasar pengerjaan tugas akhir. Teori-teori ini diambil dari berbagai literatur. Teori yang akan dijelaskan antara lain tentang teori pemampatan data, algoritma Huffman, algoritma LZW, dan transformasi Burrows Wheeler.

Analisis berisikan hasil analisis mengenai optimasi yang akan dilakukan pada kedua algoritma. Selain itu akan dianalisis kelemahan kedua algoritma, analisis optimasi yang dilakukan pada kedua algoritma, serta analisis perangkat lunak yang akan dibuat.

Perancangan dan implementasi berisikan rancangan perangkat lunak berdasarkan analisis perangkat lunak yang telah dibuat.

Pengujian dan evaluasi berisikan penjelasan tentang hasil pengujian yang dilakukan, antara lain pengujian perangkat lunak yang terdiri dari lingkungan pengujian, tujuan pengujian, data uji, kasus, uji, hasil uji serta pengujian algoritma sebelum maupun sesudah dioptimasi.

Penutup berisikan kesimpulan tugas akhir serta saran untuk pengembangan tugas akhir.

9 Penjadwalan

Penjadwalan dibawah ini berisikan *timeline* rencana kerja pembuatan tugas akhir pada semester II/2010-2011. Target kelulusan ialah bulan Juli 2011, sehingga tugas akhir harus diselesaikan sebelum bulan Mei 2011.

	Januari				Februari				Maret				April				Mei			
	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4
Studi Literatur																				
Analisis																				
Perancangan																				
Implementasi																				
Pengujian dan Evaluasi																				
Kesimpulan dan Saran																				

10 Referensi

[HAN03] Hankerson, Darrel. (2003). Introduction to Information Theory and Data Compression 2nd Edition. CRC.

[HUF52] Huffman, D. A. (1952). A Method for the Construction of Minimum Redundancy Codes. IRE.

[RAD09] Radescu, Radu. (2009). Transform Methods Used In Lossless Compression of Text Files. University of Bucharest.

[SAL07] Salomon, David. (2007). Data Compression: The Complete Reference 4th Edition. Springer-Verlag.

[SIM08] Simpson, Jamie. (2008). Words with Simple Burrows-Wheeler Transform. Curtin University of Technology.

[WIT99] Witten, Ian H. (1999). Managing Gigabyte: Compressing and Indexing Document and Images. Kaufmann Publisher.

[ZIV77] Ziv, Lempel. (1997). A Universal Algorithm For Sequential data Compression. IEEE.